

Identificação de Caminhos Disjuntos em Grafos Direcionados utilizando Fluxo Máximo

Adriano Araújo Domingos dos Santos¹

¹Instituto de Ciências Exatas e Informática – Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

R. Cláudio Manoel, 1162 – Savassi - 30.140-100 – Belo Horizonte – MG – Brazil

{adriano.domingos}@sga.pucminas.br

1. Implementação

A solução desenvolvida consiste na implementação do algoritmo de Dinic para cálculo de fluxo máximo em grafos direcionados, estendido para a identificação de caminhos disjuntos em arestas entre um par origem-destino. A aplicação foi desenvolvida em Java sem nenhuma dependência externa, salvo para os testes unitários que não são necessários para sua execução.

1.1. Algoritmo de Dinic

O algoritmo de Dinic foi implementado em sua formulação clássica, composta por duas fases principais: a construção de um grafo de níveis (*Level Graph*) por meio de uma BFS a partir da origem, e a busca de caminhos aumentantes no grafo de níveis através de uma DFS (*Blocking Flow*). A estrutura residual do fluxo é armazenada em vetores planos de inteiros para capacidade residual (`cap`), fluxo atual (`flow`) e vértice destino (`to`). Cada aresta original i é mapeada para dois índices consecutivos na estrutura residual: $2i$ para a aresta direta e $2i + 1$ para a aresta reversa, permitindo a atualização do fluxo em ambos os sentidos utilizando o operador XOR. A otimização de ponteiro atual (vetor `it`) é empregada na DFS para evitar a varredura repetida de arestas já saturadas, garantindo a complexidade $\mathcal{O}(E\sqrt{V})$ para o caso de capacidades unitárias.

1.2. Caminhos Disjuntos

A identificação de caminhos disjuntos em arestas é realizada através de uma redução para o problema de fluxo máximo. Dado um grafo direcionado $G = (V, E)$ e um par origem-destino (s, t) , cada aresta recebe capacidade unitária, transformando G em uma rede de fluxo. O algoritmo de Dinic é então executado para encontrar o fluxo máximo de s para t . Como cada aresta possui capacidade 1, o valor do fluxo máximo encontrado corresponde exatamente ao número máximo de caminhos disjuntos em arestas entre s e t . Após o cálculo do fluxo, os caminhos são extraídos percorrendo-se o grafo a partir de s , seguindo arestas com fluxo positivo, e marcando arestas já utilizadas para garantir a disjunção entre as rotas. Ao final, a implementação exibe a quantidade de caminhos disjuntos encontrados e lista individualmente cada caminho como uma sequência de vértices da origem ao destino.

1.3. Representação *Forward-Star*

A representação *Forward Star* é uma estrutura de dados baseada em vetores contíguos, que serializa a topologia do grafo em três vetores primitivos unidimensionais:

`pointers`, `targets` e `weights`. Ela foi escolhida por conta de seu acesso rápido à adjacência dos vértices e excelente uso da localidade de cache.

Na implementação desenvolvida, a construção do grafo foi estritamente desacoplada de sua representação final através do padrão de projeto *Builder*. Durante a fase de construção, as arestas são inseridas em vetores temporários paralelos. Ao finalizar a instanciação, o construtor ordena as arestas por vértice de origem, compacta os destinos no vetores finais `targets` e `weights` de tamanho $|E|$, e preenche o vetor `pointers` de tamanho $|V| + 1$. Cada índice v em `pointers` armazena o índice inicial de seus vizinhos no vetor `targets`, permitindo que a varredura da vizinhança $\Gamma(v)$ ou dos sucessores $\Gamma^+(v)$ de qualquer vértice ou o cálculo do seu grau sejam realizados em tempo $\mathcal{O}(1)$.

1.4. Gerador de Grafos Sintéticos

Para viabilizar a análise de desempenho em cenários massivos e controlados, foi desenvolvida uma ferramenta própria de geração de grafos sintéticos. O gerador é capaz de instanciar topologias direcionadas e não direcionadas, parametrizadas pela quantidade exata de vértices ($|V|$), arestas ($|E|$) e requisitos de conectividade.

A fim de satisfazer a conectividade exigida (Simplesmente ou Fortemente Conexo), a estratégia de construção foi dividida em três etapas fundamentais:

1. **Garantia de Conectividade Base:** Inicialmente, um vetor contendo todos os identificadores de vértices é embaralhado de forma aleatória utilizando o algoritmo de *Fisher-Yates*. Em seguida, itera-se sobre esse vetor conectando o vértice i ao $i + 1$, formando um caminho Hamiltoniano que garante a alcançabilidade de todos os vértices. Para grafos fortemente conexos, uma aresta de fechamento (do último vértice para o primeiro) é adicionada, transformando o caminho em um ciclo.
2. **Preenchimento de Densidade:** Para atingir a cardinalidade $|E|$ desejada, o algoritmo entra em um laço de amostragem uniforme, sorteando pares de vértices (u, v) aleatoriamente. Para assegurar que o grafo resultante seja um grafo simples (sem arestas múltiplas), as conexões geradas são validadas contra um conjunto de dispersão (`HashSet`), que armazena as arestas para buscas em tempo $\mathcal{O}(1)$ utilizando empacotamento de bits para diminuir o custo de armazenamento.
3. **Ponderação:** Após a definição da topologia, a matriz de pesos é populada. Cada aresta recebe um valor inteiro sorteado mediante uma distribuição uniforme dentro do limite parametrizado pelo usuário.

Essa abordagem garante que as instâncias geradas para o experimento sejam conexas, livres de laços e arestas paralelas.

2. Metodologia

Para avaliar a eficácia e a eficiência da implementação, foi desenvolvido um script automatizado de *benchmarking*. O experimento foi desenhado para testar os limites da estrutura e do algoritmo utilizando concorrência (*threads*) para acelerar a execução das múltiplas tentativas.

2.1. Design do Experimento e Parâmetros

Para garantir a significância estatística e mitigar anomalias causadas por processos de *background* do sistema operacional ou pelo *Garbage Collector* da JVM, cada cenário único foi executado em 10 repetições independentes. Os parâmetros estabelecidos foram:

- **Topologia do Grafo:** Foram avaliados grafos sintéticos direcionados sob duas conectividades: *Fortemente Conexo* e *Euleriano*.
- **Tamanho do Grafo:** Foram gerados grafos em quatro escalas crescentes de magnitude. Os pares de vértices ($|V|$) e arestas ($|E|$) utilizados foram: (1.000, 100.000), (20.000, 2.000.000), (500.000, 50.000.000) e (1.000.000, 100.000.000).

A combinação ortogonal de todas essas variáveis (2 tipos $\times$ 4 tamanhos) executada 10 vezes resultou em um espaço amostral de 80 execuções documentadas.

2.2. Fluxo de Execução e Coleta de Dados

Para cada iteração de teste, o fluxo seguiu um controle rigoroso para isolar os tempos:

1. **Geração:** Instanciação do grafo sintético.
2. **Construção da Estrutura:** Transcrição das arestas geradas para a estrutura de dados utilizando a representação *Forward Star*.
3. **Transformação do Grafo:** Atribuição da capacidade de cada aresta igual a 1.
4. **Busca dos Distâncias:** Execução isolada do algoritmo de Dinic, encontrando o fluxo máximo do vértice 1 até o vértice n .
5. **Registro:** Coleta e exportação das métricas de desempenho.

2.3. Métricas de Avaliação

As métricas coletadas incluíram o tempo de geração do grafo, o tempo de processamento efetivo do algoritmo de Dinic, quantidade do caminhos disjuntos encontrados e o tipo do grafo.

3. Resultados

Os experimentos foram conduzidos conforme a metodologia descrita na Seção 2. A Tabela 1 apresenta os resultados consolidados para os oito cenários avaliados, com médias calculadas sobre 10 execuções independentes.

Os resultados consolidados na Tabela 1 demonstram a escalabilidade da implementação do algoritmo de Dinic para grafos de grande porte. O tempo de execução apresentou crescimento aproximadamente linear em escala log-log (Figura 1), consistente com a complexidade esperada $\mathcal{O}(E\sqrt{V})$ para capacidades unitárias. Para a instância de maior magnitude (1 milhão de vértices e 100 milhões de arestas), o tempo médio de execução foi de aproximadamente 155 segundos para ambas as conectividades, evidenciando a viabilidade prática da abordagem mesmo em cenários massivos.

Em relação à quantidade de caminhos disjuntos encontrados (coluna Caminhos da Tabela 1), observa-se que ambos os tipos de conectividade apresentaram valores próximos ao grau médio do grafo ($|E|/|V| \approx 100$), indicando alta capilaridade da topologia gerada pelo algoritmo de Fisher-Yates com preenchimento aleatório. O número de caminhos manteve-se estável ao longo das diferentes escalas, sugerindo que a estrutura dos grafos sintéticos proporciona diversidade de rotas alternativa entre origem e destino independentemente do tamanho da instância.

Tabela 1. Desempenho médio do Dinic por tipo de conectividade.

$ V $	$ E $	Conectividade	Caminhos	Tempo (ms)
1.000	100.000	Fortemente Conexo	92,90	62,30
20.000	2.000.000	Fortemente Conexo	93,80	2251,00
500.000	50.000.000	Fortemente Conexo	95,70	70.550,30
1.000.000	100.000.000	Fortemente Conexo	97,60	155.477,90
1.000	100.000	Euleriano	93,60	45,40
20.000	2.000.000	Euleriano	95,00	2334,30
500.000	50.000.000	Euleriano	98,10	76.819,90
1.000.000	100.000.000	Euleriano	91,10	154.061,90

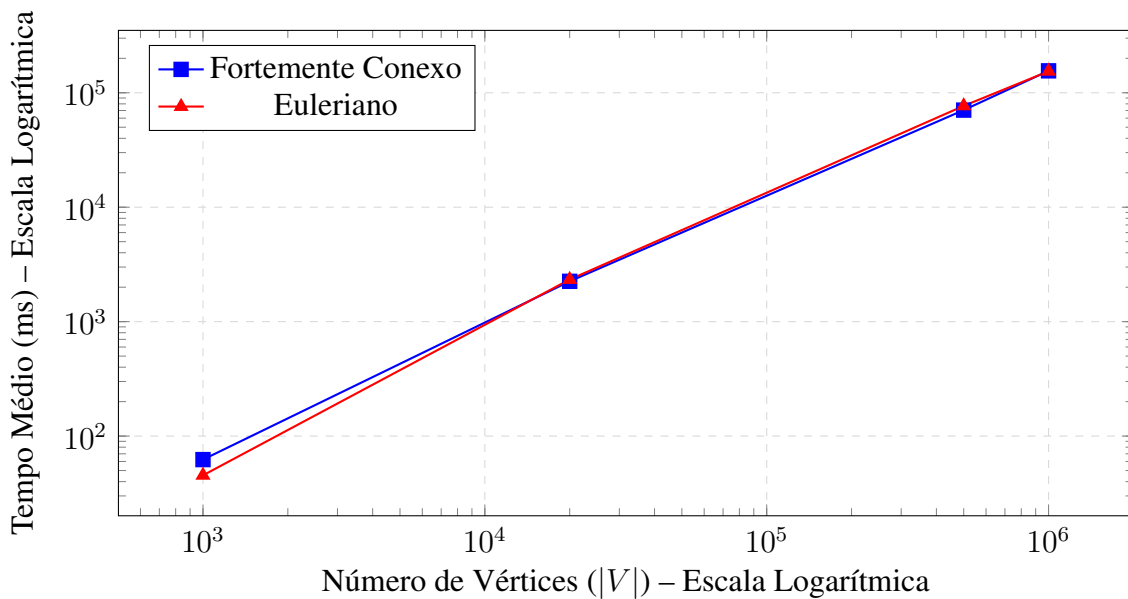


Figura 1. Crescimento logarítmico do tempo de execução do Dinic em relação ao tamanho da entrada.

4. Conclusão

A implementação do algoritmo de Dinic combinada com a representação *Forward Star* demonstrou capacidade de processar grafos com até 100 milhões de arestas em tempos viáveis para análise experimental. A redução do problema de caminhos disjuntos em arestas para fluxo máximo com capacidades unitárias mostrou-se viável, recuperando todos os caminhos sem violar a restrição de disjunção. Os resultados confirmam a escalabilidade da abordagem, habilitando sua aplicação para a identificação dos caminhos disjuntos.